

Caching

Contents

Fundamentals

1. [What a cache is](#)
2. [Where caches live](#)
3. [Measuring a cache](#)

Caching patterns

4. [Cache-aside](#)
5. [Read-through and write-through](#)
6. [Write-behind and refresh-ahead](#)

Keeping it correct

7. [Expiry and time to live](#)
8. [Invalidation](#)
9. [Consistency between cache and source](#)

Eviction

10. [Eviction policies](#)

Failure modes

11. [Stampede](#)
12. [Penetration and avalanche](#)
13. [Hot keys and large values](#)

Distributed caching

14. [Local against distributed](#)
15. [Distributing a cache cluster](#)

HTTP and CDN caching

16. [HTTP caching headers](#)
17. [CDN caching](#)

Safety

18. [Cache poisoning and safety](#)

In Java

19. [Caffeine and Spring](#)

20. [Redis in practice](#)

21. [Key and value design](#)

Reference

22. [Interview questions](#)

23. [Common mistakes](#)

24. [Quick reference](#)

1. What a cache is

A cache is a smaller, faster store holding copies of results that are expensive to produce, so that repeating the work can be skipped.

It works because real workloads are not uniform. **Temporal locality** means something just accessed is likely to be accessed again soon. **Spatial locality** means something near a recent access is likely to be next. And access distributions are usually skewed, so a small fraction of the keys accounts for a large fraction of the traffic. A cache holding that small fraction pays for itself.

What a cache costs is worth stating in the same breath, because interviewers ask for the trade rather than the benefit:

- **Staleness.** A cached copy can disagree with the source. Every design decision below is really a decision about how much disagreement is acceptable and for how long.
- **Complexity.** Two places now hold the same fact, and something has to keep them in step.
- **New failure modes.** Sections 11 to 13 exist because caches fail in ways the source does not.
- **Memory and money.** Fast storage costs more per byte than slow storage.

The question that shapes every answer: **how stale is this data allowed to be?** Data that cannot tolerate a stale copy should not use ordinary caching unless the design supplies a freshness guarantee of its own. Immutable data is the happy exception, since a value that never changes cannot go stale, which is why a content-hashed asset can be cached indefinitely. Data that tolerates minutes can use the simplest approach there is.

One principle ties the rest of this document together: **treat a cache as disposable unless the system explicitly depends on its durability.** Losing the whole cache should mean slower responses, never wrong answers or errors. Restart behaviour, warming, persistence, fallback and write-behind are all really questions about whether that still holds.

2. Where caches live

Caching happens at every layer, and naming the layers is often the opening question.

Layer	Example	Typical scope
CPU	L1, L2, L3	Nanoseconds, transparent to the application
Application memory	Caffeine, a <code>HashMap</code>	One process, lost on restart
Distributed cache	Redis, Memcached	Shared by every instance. Surviving a restart depends on the product and its configuration
Database	Buffer pool, engine-specific result caches	Inside the engine, mostly managed for us
Reverse proxy	Nginx, Varnish	In front of the application
CDN	Cloudflare, CloudFront	Near the user, geographically
Client	Browser cache, mobile store	On the user's device

Two consequences follow from the table. **The closer to the user, the cheaper the hit and the harder the invalidation**, because a copy on someone's phone cannot be reached. And **caching at several layers multiplies staleness**, since a value can be old in the CDN, older in the browser, and each layer's expiry is independent.

3. Measuring a cache

Hit ratio is hits divided by total lookups, and it is the number everyone quotes. On its own it says less than it appears to.

A rough intuition for what a cache is worth is $\text{hit ratio} \times (\text{cost of a miss} - \text{cost of a hit})$. It is not an equation to compute with, since request volume, serialisation, network latency, memory cost and eviction behaviour all matter too, but it captures the part people skip. A ninety-nine percent hit ratio on something that takes one millisecond to compute saves less than a fifty percent hit ratio on something that takes two seconds. So the useful question is not "is the hit ratio high" but "what work is this avoiding, and is that work expensive".

Worth watching alongside it:

Metric	What it tells us
Miss latency	What a miss actually costs, which sets the value of each hit
Eviction rate	Whether the cache is too small for the working set
Expiry rate against eviction rate	Whether entries leave because they aged or because memory ran out
Memory used against allocated	How close the cache is to evicting useful data
Key cardinality	Whether the key space is far larger than the cache, which caps the hit ratio

There is no target hit ratio that holds across workloads. A cache in front of a rarely repeated query may be worthless at eighty percent, and a cache in front of an expensive aggregate may be worth keeping at thirty.

One measurement trap: a cache tends to improve average latency while the tail is still dominated by misses, so p99 can barely move while the mean halves. Measuring only the mean makes a cache look better than users experience it.

4. Cache-aside

Also called lazy loading, and the most common pattern by a wide margin. The application talks to both the cache and the source, and the cache knows nothing about the database.

```
public Order findById(long id) {
    Order cached = cache.get(id);
    if (cached != null) {
        return cached;                // hit
    }
    Order order = repository.findById(id) // miss: go to the source
        .orElseThrow(OrderNotFoundException::new);
    cache.put(id, order);
    return order;
}
```

Why it dominates: only requested data is cached, so memory is not spent on things nobody reads, and a cache failure degrades into slower reads rather than an outage.

Its costs are equally characteristic. **The first request for any key always misses**, so a cold cache after a restart or deployment sends full traffic to the source. **Concurrent misses on the same key all load it**, which is section 11. And **the application owns invalidation**, since nothing else knows a write happened.

5. Read-through and write-through

These move responsibility into the cache itself, which sits in front of the source rather than beside it.

Read-through: the application asks only the cache. On a miss the cache loads from the source, stores the result and returns it. The application code has no branch in it:

```
LoadingCache<Long, Order> orders = Caffeine.newBuilder()
    .maximumSize(10_000)
    .expireAfterWrite(Duration.ofMinutes(5))
    .build(id -> repository.findById(id).orElseThrow());

Order order = orders.get(id);    // loads on miss, without an if
```

Write-through: writes go to the cache, which writes to the source before acknowledging. The cache always holds the latest value for anything it holds.

Write-through is often described as keeping cache and database consistent. That is worth qualifying: it keeps the cache from holding a value older than the last write **that went through the cache**. Any write reaching the database another way, such as a migration, a batch job or a second service, leaves the cache wrong. Consistency comes from every writer using the same path, not from the pattern itself.

The trade against cache-aside: read-through and write-through give simpler call sites and put the cache on the write path, so a cache failure now affects writes rather than only reads.

6. Write-behind and refresh-ahead

Write-behind, also called write-back, acknowledges the write once the cache holds it and writes to the source asynchronously afterwards.

It is the fastest option for write-heavy work, and it can coalesce many updates to one key into a single write to the source. It also introduces a window in which acknowledged data exists only in the cache, so a crash in that window loses it. That trade is acceptable for a view counter and not for an order, which is the interview answer: **write-behind trades durability for write latency**, and the question is whether the data can afford it.

Refresh-ahead keeps frequently used entries warm by reloading them before normal expiry, so a popular key does not present a miss to a user. Caffeine implements an access-triggered form of it rather than a background sweep over everything eligible:

```
Caffeine.newBuilder()  
    .refreshAfterWrite(Duration.ofMinutes(1))    // reload in the background  
    .expireAfterWrite(Duration.ofMinutes(10))    // hard limit if never used  
    .build(id -> repository.findById(id).orElseThrow());
```

The behaviour is worth stating precisely, because it is easy to get backwards. After one minute the entry becomes *eligible* for refresh, and nothing happens until it is next read. That read returns the **existing value straight away** and starts an asynchronous reload behind it, so no caller waits. Reads after the reload completes see the new value.

So refresh-ahead removes the latency spike, not the staleness: the first read after the interval is still served the old value, and possibly several more while the reload runs. It also costs reloads of entries that may never be read again, which is why it suits a small set of frequently read keys rather than a large sparse space.

7. Expiry and time to live

A **TTL, time to live**, is the simplest correctness mechanism there is: an entry is discarded after a fixed period whether or not anything changed.

What a TTL actually provides is a **bound on staleness**, not freshness. A five minute TTL means data is never more than five minutes out of date; it says nothing about whether the current value is correct right now. For a great deal of data that bound is the whole requirement, and no invalidation logic is needed at all.

Two expiry policies, which behave differently and get confused:

- **Expire after write** measures from when the entry was created, so an entry expires on schedule however often it is read. This bounds staleness.
- **Expire after access** measures from the last read, so a popular entry can live indefinitely. This manages memory rather than staleness, and used alone it can serve very old data.

Combining them is common: expire after access to release memory, with expire after write as the ceiling.

Add jitter to a TTL wherever entries are created together. A bulk load, a deployment or a warm-up pass gives a whole batch the same expiry, and they then all expire in the same moment, which is section 12. A random spread of a few percent removes that without changing the guarantee meaningfully. For a handful of entries created at unrelated times it buys little, so this is a rule about batches rather than about every cache.

8. Invalidation

Phil Karlton's line is quoted constantly and is worth knowing: there are only two hard things in computer science, cache invalidation and naming things. The difficulty is that invalidation requires knowing that something changed, which the cache never learns on its own.

Four approaches, most direct first:

Delete on write. After changing the source, delete the cache entry so the next read reloads it.

```
@Transactional
public void updateTotal(long id, BigDecimal total) {
    repository.updateTotal(id, total);
    cache.evict(id);           // delete, do not overwrite
}
```

One thing the annotation does not give: the eviction is not part of the transaction. If the commit fails after the evict, the cache was cleared for a change that never happened, which is harmless. If the process dies between commit and evict, the cache stays stale until its TTL. Where that ordering matters, Spring's transaction synchronization can defer the eviction until after commit.

Delete rather than update is the default worth defending. Writing the new value into the cache means two concurrent writers can interleave so the cache ends up with the older value permanently, while a delete leaves the next reader to fetch whatever the database actually holds. Updating in place is only worth it when the value is expensive to reload and writes are rare.

Versioned keys. Include a version or timestamp in the key, so a change makes every old key unreachable and no deletion is needed. Old entries age out by TTL. This suits derived data whose inputs are versioned anyway.

Event-driven invalidation. The writing service publishes a change event and every cache holding that data evicts on receipt. It suits the case where several services keep independent caches of the same data, because one event fans out to all of them, including caches the writer does not know about. It is not the only option there, since a shared distributed cache, a short TTL or versioned keys can all work instead. It also inherits the delivery guarantees of the messaging system, so a lost event means a stale cache until the TTL saves it, and keeping a TTL as a backstop is what makes that acceptable.

Expiry only. No invalidation at all, just a TTL short enough that staleness does not matter. The simplest correct answer whenever it is available, and the one people skip past too quickly.

9. Consistency between cache and source

Cache and database are two stores with no shared transaction, which is the dual write problem from the distributed systems note appearing again in a smaller form.

The ordering question is asked directly in interviews: **on a write, update the database or the cache first?**

Deleting the cache *before* writing the database leaves a window in which a concurrent reader loads the old value from the database and caches it, so the cache is stale afterwards.

Writing the database *before* deleting the cache narrows that window considerably, and does not close it. A reader that missed just before the write can still write its stale value into the cache after the delete:

```
Reader   : miss, reads old value from database
Writer   :                writes new value to database
Writer   :                deletes cache entry
Reader   :                puts old value into cache
```

The practical answer is that **database first, then delete the cache** is the better ordering, and that the remaining race still exists, with the stale value bounded by the TTL. How often it happens depends on concurrency, so the number worth stating is the maximum staleness rather than the probability. Where it genuinely cannot be tolerated, the options are a short TTL so the window closes quickly, a second delayed delete after the write, or serialising reads and writes per key. Each costs more than the problem is usually worth, and saying so is a better answer than claiming the race can be eliminated cheaply.

10. Eviction policies

A cache is finite, so something has to leave when it fills. The policy decides what.

Policy	Evicts	Suits	Weakness
LRU, least recently used	The entry unused for longest	General purpose, temporal locality	One large scan evicts the whole working set
LFU, least frequently used	The entry accessed least often	Stable popularity distributions	Old popular entries resist eviction after they stop being popular
FIFO	The oldest entry, regardless of use	Rarely the right answer	Ignores whether anything is being used
Random	An arbitrary entry	Very cheap, surprisingly competitive	No workload awareness
W-TinyLFU	Combines recency with a frequency sketch	Modern general purpose	More complex, and the sketch costs a little memory

LRU is the default because temporal locality is real and it is cheap to implement. Its known failure is a scan: a batch job reading a million rows once pushes the entire useful working set out, and the cache is cold afterwards for everyone else.

LFU fixes the scan problem, because one access does not promote an entry above something read constantly. Its own failure is the mirror image: an entry that was popular last month keeps a high count and resists eviction, so implementations age the counts.

W-TinyLFU is what Caffeine uses, and it is the modern policy worth being able to name. It keeps a small admission window for new entries, and uses a compact frequency sketch to decide whether a candidate deserves to displace the entry that would otherwise be evicted. That gives LRU's responsiveness to changing workloads with LFU's resistance to scans, at a small memory cost.

Redis is different in kind, because eviction there is a configured policy rather than a fixed one. `maxmemory-policy` selects between `noeviction` and several `allkeys` or `volatile` variants of LRU, LFU and random. The behaviour worth knowing is `noeviction`: once the configured memory limit is reached, writes needing more memory **fail** rather than older entries being evicted. That is the Redis open-source default, and the point is less about memorising a default than about choosing a policy deliberately, because failing writes is rarely what a cache is meant to do.

11. Stampede

A **cache stampede**, also called a thundering herd or dogpile, happens when a popular entry expires and every concurrent request for it misses at once, so all of them hit the source together. The source sees a spike proportional to concurrency, and the spike arrives precisely when the entry was most in demand.

Three defences, and the first is usually enough:

Single-flight loading. Only one caller recomputes; the rest wait for its result. Caffeine does this per key inside `get`, which is a good reason to use the cache's own loading method rather than a manual check and put:


```
// Concurrent callers for the same key block on one load. A manual
// get-then-put would let all of them hit the database.
Order order = cache.get(id, this::loadFromDatabase);
```

Probabilistic early expiry. Each read recomputes with a small probability that rises as expiry approaches, so one unlucky request refreshes the entry slightly early while everyone else still gets a hit. It spreads the reload without any coordination.

Serve stale while revalidating. Return the expired value immediately and refresh in the background. Latency stays flat and nobody waits, at the cost of briefly serving data past its TTL. Refresh-ahead from section 6 is the same idea applied before expiry rather than after.

12. Penetration and avalanche

Two failure modes distinct from a stampede, and interviewers often ask for all three separated.

Cache penetration is requests for keys that exist in neither the cache nor the source. Every one misses and reaches the database, and the cache never helps because there is nothing to store. It occurs naturally with sparse identifier spaces and deliberately when someone probes random identifiers.

Two defences:

- **Negative caching.** Store the absence itself, with a short TTL, so repeated lookups for a missing key are served from the cache. The short TTL matters, because the key may exist later.
- **A Bloom filter** in front of the cache, holding every key that exists in the source. It can report a false positive, which costs one wasted lookup, but never a false negative, so a key it rejects definitely does not exist. This suits very large key spaces where negative caching would itself consume too much memory. It has to be kept current, since a key created after the filter was built is rejected until the filter is updated or rebuilt, so additions need a path in.

Cache avalanche is many entries expiring at the same moment, so a burst of misses arrives together. It usually follows a bulk load, a deployment that warms a cache in one pass, or a uniform TTL applied to everything created at once.

The fix is jitter:

```
// Without jitter every entry written in this batch expires together.
Duration ttl = Duration.ofMinutes(30)
    .plusSeconds(ThreadLocalRandom.current().nextInt(300));
```

The related event is a cold cache after a restart or deployment, which produces the same shape without any expiry involved. Four ways to handle it, roughly in order of effort:

- **Lazy warming.** Accept the cold start and let traffic fill the cache. Fine when the source can absorb a burst, and the default when nothing else is needed.
- **Gradual traffic ramp.** Bring an instance into rotation slowly, so it fills its cache while serving a fraction of the load rather than all of it at once.
- **Prewarming known hot keys.** Load a known set before the instance accepts traffic, which works when the hot set is small and identifiable.
- **Avoid warming everything in one pass.** A full warm-up is itself a burst against the source, and it gives every entry the same expiry, which sets up the avalanche this section is about. Jitter matters most in exactly this case.

13. Hot keys and large values

A **hot key** is one entry taking a disproportionate share of traffic, such as a celebrity account or a global configuration value. In a distributed cache this concentrates load on the one node owning that key, and no amount of sharding helps, because it is one key.

The answers are all about spreading or shortening the path:

- **A local cache in front of the distributed one**, so most reads never leave the process. This is the most effective answer for a small number of very hot keys, at the cost of each instance holding a slightly different copy.
- **Replicating the key** under several names, such as a suffix from zero to nine, with readers choosing at random. Writes then have to update all of them.
- **Client-side request coalescing**, so concurrent local requests for the key make one remote call.

Large values cause a different problem. A multi-megabyte entry consumes bandwidth on every hit, occupies memory that many smaller entries could have used, and in Redis blocks other commands while it is serialised. Storing a computed summary rather than a whole object graph is usually the right correction, and compression helps when the values are text.

14. Local against distributed

	Local, in process	Distributed
Latency	Nanoseconds, no network	Sub-millisecond, plus a network hop
Capacity	Bounded by one heap	Bounded by a cluster
Consistency between instances	None, each instance differs	One shared namespace, though replication and read routing can still differ
Survives a restart	No	Depends on the product and its persistence settings

	Local, in process	Distributed
Failure impact	Contained to one instance	A shared dependency that can fail
Serialisation cost	None, objects are held directly	Every read and write serialises

A local cache is faster and simpler and gives every instance a slightly different view of the world. With ten instances and a one minute TTL, a value can be up to a minute stale and different on each instance, which is fine for reference data and not for anything a user might refresh and see change.

A distributed cache gives every instance the same namespace to read from, so they stop diverging by default, and it becomes a dependency of its own. How consistent that shared view actually is depends on the topology, since replication and read routing can still let two clients observe different states. If the service cannot serve requests when Redis is down, then Redis is now part of the availability calculation, and treating it as optional in the code is what keeps that from being true.

A two-level arrangement is common and worth describing: a small local cache in front of a distributed one. The local layer absorbs hot keys and the shared layer holds the bulk. The complication is invalidating the local layer, since deleting from Redis does nothing to ten process-local copies, which is what event-driven invalidation from section 8 is for.

15. Distributing a cache cluster

Spreading keys across cache nodes is the partitioning problem from the distributed systems note, and the same conclusion applies: **plain `hash(key) mod N` is the wrong choice**, because changing the node count remaps nearly every key and empties the cache at the moment a node was added to cope with load.

Consistent hashing is the standard answer. Nodes and keys are placed on a ring, each key belongs to the next node clockwise, and adding a node moves only the keys between it and its neighbour. Virtual nodes, where each physical node appears at many ring positions, even out the distribution.

Redis and Memcached take different approaches worth being able to contrast:

	Redis Cluster	Memcached
Partitioning	16384 hash slots assigned to nodes	Usually client-side, by consistent hashing
Replication	Built in, with failover	None
Data types	Strings, hashes, lists, sets, sorted sets, streams	Opaque byte values
Persistence	Optional, RDB snapshots and AOF	None
Threading	Command execution on one thread, with separate I/O threads	Multithreaded

Memcached is a simpler tool that does one thing. Redis is a data structure server that is often used as a cache, and the extra features are the reason it is usually chosen and also the reason it is easier to misuse as a database.

16. HTTP caching headers

The API note introduces these; this is the fuller set.

```
Cache-Control: public, max-age=300, stale-while-revalidate=60
ETag: "a3f5c9"
Vary: Accept-Encoding
```

Directive	Meaning
<code>max-age=n</code>	Fresh for n seconds, for any cache
<code>s-maxage=n</code>	Fresh for n seconds, for shared caches only, overriding <code>max-age</code>
<code>public</code>	Any cache may store it, including a CDN
<code>private</code>	Only the end client may store it, never a shared cache
<code>no-cache</code>	May be stored, must be revalidated before use
<code>no-store</code>	Must not be stored anywhere
<code>must-revalidate</code>	Once stale, do not serve without checking
<code>immutable</code>	Will not change before it expires, so do not revalidate
<code>stale-while-revalidate=n</code>	Serve stale for up to n seconds while refreshing in the background
<code>stale-if-error=n</code>	Serve stale for up to n seconds if the origin is failing

The distinction that catches people: `no-cache` means revalidate, `no-store` means do not keep it at all. Anything sensitive needs `no-store`.

Revalidation avoids sending the body again. The server returns an `ETag`, the client sends it back as `If-None-Match`, and an unchanged resource gets `304 Not Modified` with no body. `Last-Modified` and `If-Modified-Since` do the same with a timestamp and one second of resolution, so `ETag` is the more precise mechanism.

`stale-while-revalidate` is the header worth knowing beyond the basics, because it is section 11's serve-stale-while-refreshing applied at the HTTP layer. It hides the latency spike at expiry, and not for free: some clients are served stale content, and each refresh is an extra request arriving at the origin. The trade is usually worth taking, and it is still a trade.

Vary tells caches which request headers change the response. Omitting it where content is negotiated is a real bug: a response compressed for one client can be served to a client that cannot decompress it, or a response in one language served to a speaker of another.

17. CDN caching

A **CDN, Content Delivery Network**, is a shared cache at the network edge, close to users. It reduces latency by distance and absorbs traffic before it reaches the origin.

Three ideas do most of the work.

The cache key decides what counts as the same response. By default it is the URL, and query parameters are part of it, so a tracking parameter appended by a marketing campaign creates a separate entry for every visitor and drops the hit ratio to nothing. Configuring which parameters and headers belong in the key is the first thing to check when a CDN is not hitting.

Purge against TTL. Purging removes an object immediately and takes time to propagate across the edge network. A short TTL is often the better tool, and versioned URLs are better still: a file named with a content hash never needs purging, because a change produces a new name, which is why asset pipelines emit `app.a3f5c9.js`.

Origin shield. An intermediate caching layer between the edge locations and the origin, so a miss at fifty edges becomes one origin request rather than fifty. It is the CDN-scale version of the stampede problem.

The natural split: static assets get long TTLs and versioned names, HTML gets a short TTL or revalidation, and anything personalised is either `private` or excluded, because a shared cache serving one user's page to another is a serious bug rather than a performance issue.

18. Cache poisoning and safety

Cache poisoning is when an attacker gets a cache to store a response that is malicious or simply wrong, which is then served to other users. The cache is doing exactly what it was told; the mistake is upstream, in what it was told to treat as the same request.

Four causes, and they share a shape:

- **An incomplete cache key.** If something changes the response but is not part of the key, one stored response gets served to requests that should have received a different one. That is the `Vary` failure from section 16, and the mechanism is identical whether the result is a compressed body sent to a client that cannot read it, or an attacker-controlled value reflected into a page everyone then receives.
- **Unkeyed input reflected into the response.** A header such as `X-Forwarded-Host` used to build absolute links, but absent from the cache key, lets one crafted request poison the cached page for every later visitor.

- **Caching personalised responses.** One user's data stored in a shared cache and served to another. This is what `private` and `no-store` are for, and getting it wrong is a data breach rather than a performance problem.
- **Normalisation disagreements.** When a proxy and the origin disagree about whether two URLs mean the same thing, one form can be used to populate the entry the other will serve.

The defences follow directly: keep the cache key complete for anything that changes the response, declare `Vary` accurately, mark anything personalised `private` or `no-store`, and do not let untrusted request headers influence a response that is cacheable at all.

19. Caffeine and Spring

Caffeine is the standard in-process cache for Java, and the successor to Guava's cache.

```
Cache<Long, Order> cache = Caffeine.newBuilder()
    .maximumSize(10_000)           // entries, not bytes
    .expireAfterWrite(Duration.ofMinutes(5)) // bounds staleness
    .refreshAfterWrite(Duration.ofMinutes(1)) // background reload
    .recordStats()                 // enables hit ratio metrics
    .build();
```

`maximumSize` counts entries rather than memory, so a cache of ten thousand large objects can still exhaust a heap; `maximumWeight` with a weigher is the answer when entry sizes vary a lot. `recordStats` is worth switching on from the start, since a cache without a measured hit ratio is a guess.

Spring's cache abstraction puts this behind annotations:

```
@Cacheable(value = "orders", key = "#id")
public Order findById(long id) { ... }

@CacheEvict(value = "orders", key = "#order.id")
public void update(Order order) { ... }

@CachePut(value = "orders", key = "#order.id")
public Order createAndCache(Order order) { ... }
```

`@Cacheable` skips the method on a hit. `@CacheEvict` removes an entry. `@CachePut` always runs the method and stores the result, which is the one people choose by mistake when they wanted `@Cacheable`.

The proxy trap, again

Spring implements `@Cacheable` with a proxy, exactly as it does `@Transactional`. Calling a cached method from another method of the same class bypasses the proxy, so the cache is not consulted and not populated, with no error to show for it. The same limitation applies to `private` and `final` methods. The design patterns note covers why, and it is worth recognising as one behaviour rather than three separate quirks.

20. Redis in practice

Redis is **mostly single-threaded**: command execution happens on one thread, while networking and some background work use others in recent versions. That single execution thread is the source of several properties worth knowing. Command execution is effectively atomic, so no command sees another halfway through, and that is what makes operations such as `INCR` and `SETNX` usable as primitives. It also means **one slow command blocks everything else**, so `KEYS *` on a large database is a well known way to stall a production instance. `SCAN` exists precisely because it iterates in bounded chunks.

Commands worth having ready:

<code>SET key value EX 300</code>	set with a 300 second expiry
<code>SET key value NX EX 30</code>	set only if absent, which is a lease
<code>GET key</code>	read
<code>DEL key</code>	invalidate
<code>TTL key</code>	remaining life, -1 no expiry, -2 no key
<code>INCR key</code>	atomic counter
<code>SCAN 0 MATCH order:*</code>	iterate without blocking, unlike <code>KEYS</code>

Three operational points:

Persistence is optional and configurable. RDB takes point-in-time snapshots and AOF appends every write. A cache is disposable by the principle in section 1, so it often needs neither, and turning persistence off avoids the fork overhead that both snapshotting and AOF rewriting introduce, at the cost of losing everything on restart. Whether that cost matters is the same question as whether the cache can be treated as disposable.

Set `maxmemory` and an eviction policy deliberately. Under `noeviction`, writes that need more memory fail once the limit is reached rather than older entries being evicted, which is rarely what a cache is meant to do.

Pipelining and `MGET` cut round trips. Fetching a hundred keys one at a time is a hundred round trips, and most of the latency is the network rather than Redis.

21. Key and value design

Keys are an interface, and inconsistent ones are the reason a cache becomes impossible to reason about.

order:v2:42
user:42:orders:page:1

type, schema version, identifier
hierarchy, colon separated by convention

Four rules that hold up:

- **Namespace by type**, so keys are greppable, groupable and safe to expire in bulk by prefix.
- **Include a schema version**. When the cached shape changes, a version bump makes every old entry unreachable instead of requiring a flush, and avoids the deserialisation failures that follow a deploy.
- **Keep keys short but readable**. Every key is stored in memory, and at tens of millions of entries the key size is a real cost, while an unreadable key makes debugging much harder.
- **Bound and validate anything user-controlled that reaches a key**. The risk is not only invalid input, it is unbounded cardinality: a caller who can vary one component freely creates endless distinct keys, filling the cache with entries nobody will read again and evicting everything useful.

On values, cache **what is expensive to compute, not what is convenient to store**. A rendered summary that took four joins is a good candidate; an entity that is one indexed primary key lookup usually is not.

Serialisation is the detail that causes deployment incidents. Java's native serialisation ties cached bytes closely to class definitions. It does have compatibility rules, so not every change breaks, but working out which changes are safe is its own exercise and the failure shows up at runtime, in the middle of a rolling deploy, when two versions are reading each other's entries. JSON with tolerant reading, or an explicit versioned binary format, keeps that from happening, and it is another reason a versioned key prefix earns its place.

22. Interview questions

Each answer below is written to be said out loud as it stands. The last sentence always leads toward the next question rather than closing the topic.

"What caching pattern would you use, and why?"

Cache-aside by default. The application checks the cache, and on a miss it loads from the database and puts the value in, so only data that is actually requested gets cached and a cache failure degrades into slower reads rather than an outage. The costs are that the first request for any key always misses, so a cold cache after a deploy sends full traffic to the database, and that the application owns invalidation because nothing else knows a write happened. Read-through and write-through move that into the cache and give simpler call sites, at the price of putting the cache on the write path. Write-behind is the fastest for write-heavy work and trades durability for it, since a crash loses anything acknowledged but not yet written, so I would use it for something like a view counter and not for an order.

"How do you keep a cache consistent with the database?"

Strictly, I do not, because they are two stores with no shared transaction. The honest goal is to bound staleness and be explicit about what happens during the inconsistency window, rather than to eliminate it. The default is to write the database first and then delete the cache entry, rather than updating it, because two concurrent writers updating in place can interleave and leave the older value cached permanently, while a delete just makes the next reader fetch whatever the database holds. That ordering narrows the race but does not close it: a reader that missed just before the write can still put its stale value in after the delete. The remaining window still exists and its size is bounded by the TTL, which is the number worth quoting rather than a probability, and if it genuinely cannot be tolerated the options are a short TTL, a second delayed delete, or serialising per key, all of which cost more than the problem usually justifies.

"What is a cache stampede, and how do you prevent it?"

It is when a popular entry expires and every concurrent request for it misses at once, so they all hit the database together, and the spike arrives exactly when that key was most in demand. The simplest fix is single-flight loading, where only one caller recomputes and the rest wait for its result, which Caffeine does per key inside its loading `get`, and that is a good reason to use the cache's own loader rather than a manual check and put. Beyond that, probabilistic early expiry has each read refresh with a small probability that rises as expiry approaches, so one request refreshes slightly early and everyone else still gets a hit, and serving stale while revalidating in the background keeps latency flat at the cost of briefly serving expired data. At the HTTP layer that last one is just `stale-while-revalidate`.

"What is the difference between penetration, stampede and avalanche?"

They are three different failure modes and it is worth separating them. A stampede is one popular key expiring and many concurrent requests missing it together. Penetration is requests for keys that exist in neither the cache nor the database, so every request reaches the source and the cache can never help, which happens naturally with sparse identifiers and deliberately when someone probes random ones. The fixes for penetration are negative caching, storing the absence itself with a short TTL, or a Bloom filter in front, which can give a false positive costing one wasted lookup but never a false negative. Avalanche is many entries expiring at the same moment, usually because they were all created together with the same TTL, and the fix is jitter on the TTL so they spread out.

"LRU or LFU?"

LRU by default, because temporal locality is real and it is cheap, but its known failure is a scan: one batch job reading a million rows once evicts the whole working set and leaves the cache cold for everyone else. LFU fixes that, since a single access does not promote an entry above something read constantly, but it has the mirror problem where something popular last month keeps a high count and resists eviction, so implementations age the counts. The current answer is usually W-TinyLFU, which is what Caffeine uses: a small admission window for new entries plus a compact frequency sketch deciding whether a candidate deserves to displace what would otherwise be evicted, which gives LRU's responsiveness with LFU's resistance to scans. On Redis the policy is configuration rather than a fixed choice, and the thing to check is that `maxmemory-policy` is not left at `noeviction`, which makes writes fail instead of making room.

"Local cache or Redis?"

It depends on whether instances are allowed to disagree. A local cache is nanoseconds with no network hop and no serialisation, and every instance holds a slightly different view, so with ten instances and a one minute TTL a user refreshing can see the value change back and forth. That is fine for reference data and not for anything user-visible that changes. Redis gives one shared view, survives restarts and deploys, and becomes a dependency that can fail, so I would want the service to degrade to slower reads rather than errors when it is down. Often the answer is both: a small local cache in front of Redis to absorb hot keys, with the complication that deleting from Redis does nothing to ten process-local copies, so that layer needs event-driven invalidation or a short enough TTL that it does not matter.

"How would you handle one extremely hot key?"

No amount of sharding helps, because it is one key and it lives on one node, so the answers are about shortening or spreading the path. The most effective is usually a local in-process cache in front of the distributed one, so the vast majority of reads never leave the process, accepting that each instance may hold a slightly different copy for a short time. Beyond that, the key can be replicated under several names with a random suffix and readers choosing at random, which spreads the load across nodes at the cost of writes having to update all of them, and request coalescing so concurrent local requests make one remote call. I would also check whether it needs to be a cache entry at all, since something like global configuration read constantly and changed rarely can be held in memory and refreshed on an event.

"Why might a cache make things worse?"

A few ways, and the first is that it adds a failure mode rather than removing one. If the service cannot serve requests when the cache is down, availability has gone down rather than up, so the code has to treat a cache failure as a slow path rather than an error. Then there is staleness, since two stores now hold the same fact and something has to keep them in step, and every extra layer multiplies it because a value can be old in the CDN, older in the browser, and each expiry is independent. There is also the case where it simply does not pay: the value of a cache is roughly the hit ratio times the cost of a miss minus the cost of a hit, so a ninety-nine percent hit ratio on something that takes a millisecond saves less than a fifty percent hit ratio on something that takes two seconds. And a cache can hide a problem worth fixing, where a query that needed an index gets a cache instead and the underlying cost is still there for every miss.

23. Common mistakes

- Caching data that cannot tolerate any staleness, then adding invalidation logic complex enough to be its own source of bugs.
- Updating the cache in place on a write rather than deleting the entry, so two concurrent writers can interleave and leave the older value cached permanently.
- Deleting the cache before writing the database, which widens the window for a reader to store the stale value.
- Applying a uniform TTL with no jitter, so entries created together expire together and produce a burst of misses.
- Treating a cache as always available, so an outage in it becomes an outage in the service instead of slower responses.
- Calling a `@Cacheable` method from another method in the same class, which bypasses the proxy so nothing is cached and nothing reports an error.
- Using `@CachePut` where `@Cacheable` was meant, which runs the method every time and caches the result, giving no hits at all.
- Leaving Redis at the default `noeviction` policy, so writes start failing when memory fills instead of older entries being evicted.
- Running `KEYS *` against a production Redis, which blocks command execution while it scans. `SCAN` exists for this.
- Caching a whole entity that costs one indexed primary key lookup, rather than the expensive derived value that took several joins.
- Serialising with Java's native serialisation, so a class change breaks every instance still running the previous version during a rolling deploy.
- Omitting a schema version from the key, so a change to the cached shape needs a flush or produces deserialisation failures after a deploy.
- Leaving tracking query parameters in a CDN cache key, which creates a separate entry per visitor and drops the hit ratio to nothing.

- Using `no-cache` when `no-store` was meant. The first permits storage and requires revalidation; only the second keeps it out of caches.
- Judging a cache by hit ratio alone, without asking what a miss actually costs.

24. Quick reference

Term	Meaning
Cache-aside	Application checks the cache, loads from the source on a miss, then stores
Read-through	The cache loads from the source itself on a miss
Write-through	Writes go through the cache to the source before acknowledging
Write-behind	Acknowledge from the cache, write to the source asynchronously
Refresh-ahead	Reload a popular entry before it expires, so no user sees the miss
TTL	Bounds how stale data can be. Says nothing about being fresh now
Expire after write	Measured from creation. Bounds staleness
Expire after access	Measured from last read. Manages memory, not staleness
Negative caching	Cache the absence of a value, with a short TTL
LRU	Evict least recently used. Cheap, defeated by a scan
LFU	Evict least frequently used. Resists scans, ages badly without decay
W-TinyLFU	Admission window plus frequency sketch. What Caffeine uses
<code>noeviction</code>	Redis default. Writes fail when memory is full rather than evicting
Stampede	One key expires, many concurrent requests miss it at once
Penetration	Requests for keys absent from both cache and source
Avalanche	Many entries expiring together, causing a burst of misses
Hot key	One entry taking a disproportionate share of traffic
Single-flight	One caller recomputes, the rest wait for that result
Consistent hashing	Ring placement, so adding a node moves roughly a <code>1/N</code> share of keys
<code>max-age</code> against <code>s-maxage</code>	Any cache, against shared caches only
<code>no-cache</code> against <code>no-store</code>	Revalidate before use, against do not store at all
<code>stale-while-revalidate</code>	Serve the stale copy while refreshing behind it
<code>Vary</code>	Which request headers change the response, and so the cache key
ETag	Version tag, returned as <code>If-None-Match</code> , answered with 304

Term	Meaning
Origin shield	A CDN layer that collapses many edge misses into one origin request

Symptom	Likely cause
Hit ratio near zero	Key cardinality far exceeds capacity, or the key includes something unique
A burst of database load every few minutes	Uniform TTL with no jitter, so entries expire together
Database spike on one popular key	Stampede, with no single-flight loading
Constant misses for identifiers that do not exist	Penetration, needing negative caching or a Bloom filter
One cache node saturated while others idle	A hot key, which sharding cannot split
Cache empties whenever a node is added	Modulo hashing instead of consistent hashing
A method annotated <code>@Cacheable</code> never caches	Called from inside the same class, so the proxy is bypassed
Writes failing once memory fills	Redis <code>maxmemory-policy</code> left at <code>noeviction</code>
Redis latency spikes with no traffic change	A blocking command such as <code>KEYS</code> , or a snapshot fork
Deserialisation errors after a deploy	Cached shape changed with no version in the key
Stale value served permanently	Cache updated in place on write, and two writers interleaved
CDN hit ratio collapses after a campaign	Tracking query parameters included in the cache key
Average latency improves, p99 does not	The tail is dominated by misses, which the mean hides